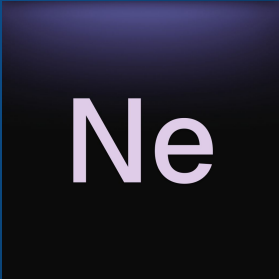


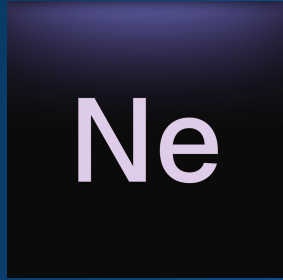
Low Level C++ Theory & Applications

Amlal El Mahrouss

The logo for Ne, featuring the letters 'Ne' in a white, sans-serif font centered within a dark purple square with a subtle gradient and a slight shadow.

Ne

I'm Amlal from Ne.app.



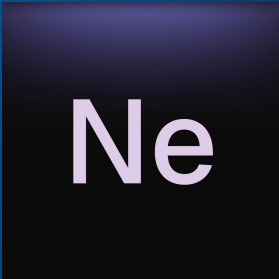
Email: amlal@ne-app.eu

Purpose of the talk:

→ In this talk we're going to layout techniques and decisions to take in order to successfully develop a low-level system in modern C++.

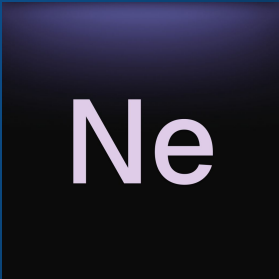
The standards involved:

→ C++17 and later.

The logo for the Ne programming language, featuring the letters 'Ne' in a white, sans-serif font on a dark purple square background.

On References:

- Unreference claims are not included in this talk for the sake of accuracy.
- P.S: These slides are available on GitHub. [publications-and-more/talk](#).
- Thank you.

The logo consists of a dark blue square with a subtle gradient. Inside the square, the letters "Ne" are written in a white, sans-serif font. The "N" is slightly larger than the "e".

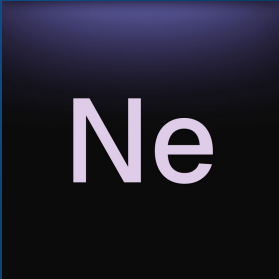
Ne

I: The ‘Core’ Principles.

Amlal El Mahrouss

Observation: What we will cover in this talk.

- Why C++ over C? (In a 'N%' percent of times)
- How C++ evolved (in a good way).
- What to learn from C++ in your codebase.

The logo for Ne, consisting of the letters 'Ne' in a white, sans-serif font, centered within a dark blue square with a subtle gradient and a slight shadow effect.

Ne

II: Common RAII Patterns.

Amlal El Mahrouss

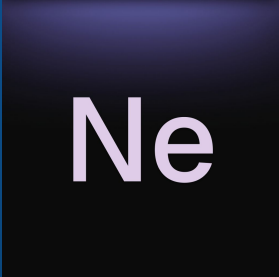
When to use RAII?

When using non-freestanding C++ RAII is mostly a ‘no-brainer’:

→ `std::unique_ptr`.

→ `std::shared_ptr`.

However, one may judge to roll its own container when deemed necessary, such as Open C++ Libraries, ‘`tracked_ptr`’, or Boost’s custom expansions of shared pointers operations such as `delete_ptr`.

A dark blue square with a subtle gradient and a thin white border, containing the letters 'Ne' in a white, sans-serif font.

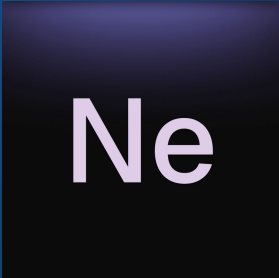
Ne

When to use RAII?

Per ANT resource handling in HAL.dll: It's architecture makes use of the RAII pattern to manage resources that are deemed to be eligible for such operations.

For example, file descriptors that are used by a routine are automatically freed after being run. Or HAL entries being cleared outside of memory for memory reasons.

Such operations are: Load, Unload, Call, Add, Remove.

A dark blue square with a gradient, containing the letters "Ne" in a white, sans-serif font.

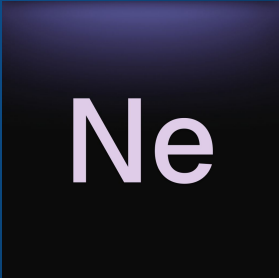
When to use RAII?

Another example: A resource allocator for a graphics engine.

This is a non-trivial operation, where the complexity of the algorithm may be exponential here. Thus per RAII, we will need to perform caching and implement a pool allocation algorithm.

This helps for:

- Linearity of the search.
- Searching time per algorithm complexity.

The logo for Ne, featuring the letters 'Ne' in a white, sans-serif font, centered on a dark blue square background with a subtle gradient.

III: Several RAII Patterns.

Amlal El Mahrouss

The OwnPtr, shared_ptr, and auto_ptr.

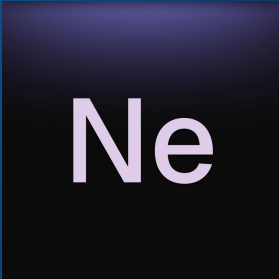
A type introduced in WebKit, back when C++0x wasn't formalized yet.

ANT and NeSystem have their own RAII primitives.

And so does many C++ software...

Even some C based software does (thinking about the GCC extensions for that purpose)

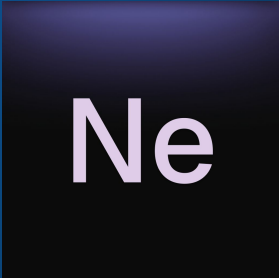
Source: [Implementing RAII in pure C? - Stack Overflow](#)

The logo for Ne, consisting of the letters 'Ne' in a white, sans-serif font, centered on a dark blue square background with a subtle gradient.

Design Matters!

After shipping and maintaining several software projects, I've come to this conclusion, and RAII containers are a prime example of this. What was once started with `auto_ptr`, had to be scaled to several refactorors (unique, shared pointers) post C++11 due to the demand of more demanding containers.

I would love to talk more about how could one would design such software in the long-term, but I'm afraid It's not the scope of the talk.

The logo consists of a dark blue square with a subtle gradient. Inside the square, the letters "Ne" are written in a white, sans-serif font. The "N" is slightly larger than the "e".

Ne

IV: Applications and Examples of RAII.

Amlal El Mahrouss

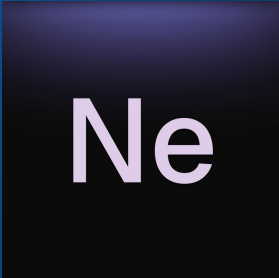
Example one: A Telnet Server

```
$ telnet towel.blinkenlights.nl
```

Now... One shall handle, the TUI, resources allocation, logging, etc...

What happens if the resource is busy? What happens if the NIC is being plugged off? Or the router crashes?

That's why RAII is crucial, it helps handles those cases!

A dark blue square with a gradient, containing the white text "Ne" in a serif font.

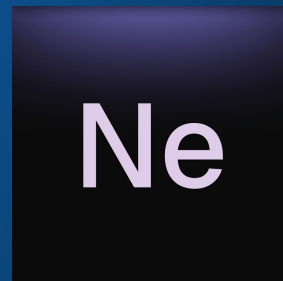
Example one: Ne.app allocators and abstracting them into a RAII container.

The following snippet is written in C:

```
VoidPtr x = MmAllocHeap(sz, flags, ...);
```

This compiles in C, but could be improved for ‘higher level applications’*

[1] An application that does not require manual management
Of the process’ memory.



V/VI: Conclusion.

Amlal El Mahrouss